Mittuniversitetet
MID SWEDEN UNIVERSITY

Försättsblad Prov Original

Kurskod	Provkod	Tentamensdatum
DT019G	T107	2025-08-18
Kursnamn	Datateknik GR (A), Objektbaserad programmering i C++	
Provnamn	Skriftligt teoriprov	
Ort	Sundsvall	
Termin		
Ämne		

6895642268

Utskriven 2025-09-01 kl. 14:22:33

i



Mittuniversitetet
MID SWEDEN UNIVERSITY

Tentamen

DT019G Objektbaserad programmering i C++
DT026G Objektbaserad programmering

Jimmy Åhlander

Instruktioner

Tid	3 timmar
Resurser	Laptop, mus, penna, sudd och tillhandahållet papper
Poäng	100
Preliminära betygsgränser	G 65 %
Allmänt	För uppgifter av typen flersvar ges poäng för varje korrekt markerat svar och minuspoäng för varje inkorrekt markerat svar. Minimum för en enskild uppgift är alltid noll poäng. Poängfördelningen är inte alltid jämn mellan alternativen i en uppgift. Tentan är anonymiserad. Skriv aldrig ditt namn, personnummer eller annan identifierande information i dina svar. Om inget annat är angivet så ska uppgifterna alltid besvaras inom kontexten för C++. Vid tekniska problem, kontakta alltid tentamensvakt. Stäng inte av eller starta om datorn utan uppsikt om du inte redan lämnat in tentamen. InsperaScan används ej för denna tentamen.

Mittuniversitetet
Institutionen för data- och elektroteknik (DET)

Jimmy Åhlander, 010 - 142 86 90

1(a) Vilka av följande rader deklarerar giltiga konstruktorer för klassen Foo?

Välj ett eller flera alternativ:

- ☐ Foo();
- ☐ Foo(Bar b);
- ☐ Bar(Foo f);
- ☐ Foo construct(int i);
- ☐ Foo(bar b, ken m);
- ☐ ~Foo();

Totalpoäng: 5

1(b) Vilka av följande påståenden är sanna efter att följande kod exekverat?

```
int *p, *q = new int(3);
p = q;
*q = *p;
delete q;
p = nullptr;
```

Välj ett eller flera alternativ:

- ☐ p blir en dangling pointer.
- ☐ q blir en dangling pointer.
- ☐ p blir initialt en wild pointer.
- ☐ q blir initialt en wild pointer.
- ☐ Odefinierat beteende uppstår.
- ☐ En minnesläcka uppstår.
- ☐ Bara ett av de tre allokerade talen i `q` blir frigjort.

Totalpoäng: 5

1(c) Vad kännetecknar en defaultkonstruktor?

Välj ett alternativ:

- ○ Den kan anropas utan argument.
- ○ Det är den första konstruktorn som definierats för en klass.
- ○ Den kan endast implementeras implicit (syntetiserad).
- ○ Den kan konvertera parametrarnas datatyp (maximalt en gång), t.ex. int till double.
- ○ Den skapar tomma objekt.

Totalpoäng: 5

1(d) Vad sker vid följande kodrad?

```
Maze m;
```

Välj ett alternativ:

- ○ Ett objekt m av Maze skapas med defaultkonstruktorn.
- ○ Ett objekt m av Maze skapas med den syntetiserade kopieringskonstruktorn.
- ○ En funktion m som returnerar en Maze deklareras.
- ○ En klass Maze m deklareras.

Totalpoäng: 5

1(e) Vad innebär `const` i kontexten för en medlemsfunktionen deklarerad som `int sum(int a[]) const;`?

Välj ett alternativ:

- ○ Funktionen kan inte ändra parametern `a`.
- ○ Funktionen kan inte ändra *elementen* i arrayen `a`.
- ○ Returvärdet måste vara konstant.
- ○ Funktionen kan inte ändra datamedlemmarnas värden.
- ○ Funktionen kan endast arbeta med konstanter, inte variabler.

Totalpoäng: 5

1(f) Vilken nivå av undantagssäkerhet garanterar att inga resurser läcker och att alla objekt förblir i ett giltigt, om än förändrat, tillstånd efter att ett fel kastats?

Välj ett alternativ:

- ○ No-throw guarantee
- ○ No exception guarantee
- ○ Basic exception guarantee
- ○ Medium exception guarantee
- ○ Strong exception guarantee

Totalpoäng: 5

1(g) Vilka av följande deklarationer deklarerar medlemmar till Foo som är en del av *The Big Three* (rule of three)?

Välj ett eller flera alternativ:

- ☐ Foo();
- ☐ Foo(const Foo&);
- ☐ Foo(size_t);
- ☐ explicit Foo(int);
- ☐ ~Foo();
- ☐ Foo& operator=(const Foo&);
- ☐ Foo& operator+=(const Foo&);
- ☐ ostream& operator<<(ostream&, const Foo&);
- ☐ istream& operator>>(istream&, Foo&);

Totalpoäng: 5

1(h) Vad skrivs ut av följande kod?

```
std::string s = "snickers";
std::cout << s.find("ick");
```

Välj ett alternativ:

- ○ true
- ○ false
- ○ 0
- ○ 1
- ○ 2
- ○ 3
- ○ ick
- ○ snickers

Totalpoäng: 5

1(i) Se kodraden nedan:

```
int i = 3;
```

Vad blir resultatet av följande operationer?

Matcha ihop värdena:

	3	Adressen till pekaren	Adressen till heltalet	Inget av de andra alternativen
i	○	○	○	○
*i	○	○	○	○
&i	○	○	○	○
&*i	○	○	○	○
*&i	○	○	○	○

Totalpoäng: 5

1(j) Anta att du har följande vektor v:

```
std::vector<int> v = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Anta nu att du tar bort ett element från v, t.ex. med **`std::vector::pop_back`**. Vilken *kapacitet* (*capacity*) kommer vektorn ha efter borttagningen?*

Välj ett alternativ:

- ○ 0
- ○ 1
- ○ 9
- ○ 10
- ○ 11
- ○ 15

* *Svaret är implementationsspecifikt men är densamma för MSVC, G++ och CLANG.*

Totalpoäng: 5

1(k) Vilken av följande kodrader matchar den senare av de två kodraderna nedan?

```
std::string s;
s += s += s;
```

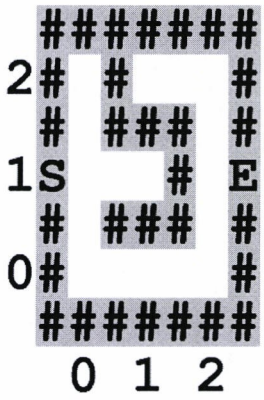
Välj ett alternativ:

- ○ operator+=(s).operator+=(s);
- ○ operator+=s->operator+=s
- ○ operator+=.(s)->operator+=
- ○ s.operator+=(s.operator+=(s));
- ○ s->operator+=(s->operator+=(s));
- ○ s.operator+=s.operator+=s;

Totalpoäng: 5

1(I) Följande bild visar en 3x3-labyrint bestående av 9 noder genererad med *depth-first search* (DFS). Vid vilken koordinat startade DFS genereringen av labyrinten?

Koordinaterna är uttrycka på formen (x, y), motsvarande (kolumn, rad).



Välj ett alternativ:

- (0,0)
- (0,1)
- (0,2)
- (1,0)
- (1,1)
- (1,2)
- (2,0)
- (2,1)
- (2,2)

Totalpoäng: 5

i

Inför uppgifter där kod tolkas

I de kommande uppgifterna tolkar och evaluerar du kod som någon annan producerat. Läs uppgifterna noga då målet kan skilja sig mellan dem. Det kan handla om att identifiera brister i koden eller att besvara en särskild fråga.

I vissa fall anropas funktioner vars kod inte presenteras. I de fallen kan du anta att de anropade funktionerna fungerar ändamålsenligt och returnerar den mest rimliga datatypen givet kontexten.

2(a) Klassen **`linked_list`** representerar en länkad lista vars noder håller flyttal. Objekt av klassen håller två pekare: head och tail som leder till första respektive sista noden i listan. Varje nod håller två pekare: next och prev som leder till nästa nod (närmare svansen) respektive föregående nod (närmare huvudet) samt ett flyttal **`value`**. Nedan följer koden för medlemsfunktionen **`push_back`** som lägger till en ny nod längst bak i listan.

```
void linked_list::push_back(double value) {
    node* new_node = new node(value);
    new_node->value = value;
    if (is_empty()) {
        head = new_node;
        tail = new_node;
    }

    new_node->prev = tail;
    tail->next = new_node;
    tail = new_node;
}
```

Vilka problem har koden?

Välj ett eller flera alternativ:

- ☐ Fallet där listan är tom hanteras inkorrekt eller inte alls.
- ☐ Fallet där listan redan håller exakt en nod hanteras inkorrekt eller inte alls.
- ☐ Fallen där listan redan håller flera noder hanteras inkorrekt eller inte alls.
- ☐ Nodens flyttal tilldelas aldrig i vissa situationer.
- ☐ Tilldelningen av value är redundant.
- ☐ I vissa fall kan minnesläckor uppstå, t.ex. om push_back anropas flera gånger på samma lista.

Totalpoäng: 5

2(b) Den rekursiva funktionen **`hanoi`** kan användas för att visa vilka drag som krävs för att lösa tornen i hanoi-problemet med **`n`** diskar. En stödfunktion **`move`** används för att skriva ut varje enskilt drag till skärmen.

```
void move(char src, char dst) {
    std::cout << "Move from " << src << " to " << dst << std::endl;
}

void hanoi(int n, char src, char via, char dst) {
    hanoi(n - 1, src, dst, via);
    move(src, dst);
    hanoi(n - 1, src, dst, via);
}
```

Vilka problem har koden?

Välj ett eller flera alternativ:

- ☐ Funktionen ska vara svansrekursiv (tail recursive) för att följa lösningsalgoritmen.
- ☐ Argumenten är felaktiga i det första rekursiva funktionsanropet.
- ☐ Argumenten är felaktiga i det andra rekursiva funktionsanropet.
- ☐ Stödfunktionen **`move`** är inkorrekt implementerad och kommer skriva ut felaktiga drag som ett resultat.
- ☐ Det saknas ett basfall (base case).
- ☐ En stack overflow kommer uppstå vid körning.

Tips:

- Ett typiskt anrop skulle kunna vara: **`hanoi(3, 'A', 'B', 'C');`**.

Totalpoäng: 5

i

Inför uppgifter där kod kompileras

I de kommande uppgifterna skriver du kod som kompileras, testkörs och valideras genom ett antal tester. För dessa uppgifter:

- Används C++14 (gcc-5 5.1.1). Observera att funktionalitet exklusiv till C++17, C++20 eller nyare därför inte kan användas. I normalfallet bör detta inte påverka dig nämnvärt.
- Skrivs all kod i motsvarigheten till en fil. Det finns ingen uppdelning i header- och cpp-filer. Du kan skriva dina deklarationer och definitioner i samma textfält eller helt enkelt skriva skriva dina definitioner inline.
- Måste samtliga testfall i varje uppgift valideras för att erhålla full poäng. Validerade tester innebär dock inte per automatik att du får poäng, exempelvis ger inte hårdkodad utdata poäng.
- Måste du endast göra utskrifter där du instruerats att göra så. Extra utskrifter som inte efterfrågats t.ex. "Hej och välkommen till mitt program! Skriv in ett tal" kommer invalidera de automatiska testerna.
- Tillåts inte globala variabler. Globala konstanter är acceptabla.

3 Implementera en klass MyInt. Ett objekt i klassen MyInt ska kunna hålla ett enda dynamiskt allokerat heltal. MyInt har därför endast en medlemsvariabel.

MyInt ska ha följande medlemmar:

- En medlemsvariabel **`ptr`** av typen int*.
- En parametriserad konstruktor som tar **`num`** av typen int som inparameter.
- En destruktor.
- En överlagrad tilldelningsoperator (=). Kopieringen måste vara djup.
- En funktion **`set`** som sätter värdet på talet **`ptr`** pekar på.
- En funktion **`get`** som returnerar värdet på talet **`ptr`** pekar på.

Medlemmarna ska inkapslas enligt konvention. Minnesläckor måste undvikas. Du behöver inte visa hänsyn för regeln the big three, d.v.s. du behöver inte implementera någon annan funktion än de listade ovan för full poäng.

För att testa din klass skapar du två objekt **`i`** och **`j`** samt anropar funktioner enligt kommentarerna nedan.

Testfall #	Input	Förväntad output
1	5 11 27	5 11 27
2	1 2 3	1 2 3
3	19 21 30	19 21 30

```
// kom ihåg dina includes
int main() {
    int a, b, c;
    std::cin >> a >> b >> c;

    // skapa ett objekt i av MyInt genom att anropa konstruktorn med a som argument
    // anropa get på i och skriv ut till cout, inkl. ny rad

    // anropa set på i med b som argument
    // anropa get på i och skriv ut till cout, inkl. ny rad

    // skapa ett till objekt j av MyInt genom att anropa konstruktorn med c som argument
    // använd tilldelningsoperatorn för att tilldela i värdet från j. (båda ska nu ha värde c)

    // anropa get på i (obs! i, inte j) och skriv ut till cout, inkl. ny rad

    return 0;
}
```

Testkod

Totalpoäng: 15

4 Implementera en rekursiv funktion som konverterar ett decimaltal till en binär sträng. Strängen ska returneras av funktionen.

Exempelvis:

5 → "101".

42 → "101010".

Testfall #	Input	Förväntad output
1	0	0
2	1	1
3	5	101
4	42	101010

```
// kom ihåg dina includes

... decimal_to_binary(...) {
    // din lösning här
}

int main() {
    // läs in talet från stdin och skriv ut den returnerade strängen till stdout
    return 0;
}
```

Testkod

Observera:

- Funktionen måste returnera en std::string med den färdiga strängen.
- Lösningen måste vara rekursiv där rekursionen är av väsentlig betydelse för lösningen.
- Loopar får inte användas för att kringgå rekursionen.
- Globala variabler får inte förekomma.

Tips:

- Du kan tillämpa algoritmen som introducerades i kursen Datateknik GR (A) *Grundläggande datavetenskap* för ändamålet. Du använder då heltalsdivision och noterar tillhörande resten för att konstruera den binära strängen. Fortsätt dividera tills att kvoten blir ett.
- Funktionen std::to_string kan vara användbar.

Totalpoäng: 15